CSA1737 – ARTIFICIAL INTELLIGENCE

LIST OF EXPERIMENTS

1. Write a python program to implement Simple Calculator program?

AIM:

To write a Python program to perform basic arithmetic operations such as addition, subtraction, multiplication, and division.

ALGORITHM:

1. Start the program.
2. Get two numbers from the user.
3. Display the arithmetic operation choices.
4. Get the user's choice.
5. Perform the selected operation using if-elif-else.
6. Display the result.
7. Stop the program.

CODE:

main.py

```
1
2  a = float(input("Enter first number: "))
3  b = float(input("Enter second number: "))
4  op = input("Enter operator (+, -, *, /): ")
5  if op == "+":
6      print("Result:", a + b)
7  elif op == "-":
8      print("Result:", a - b)
9  elif op == "*":
10     print("Result:", a * b)
11 elif op == "/":
12     if b != 0:
13         print("Result:", a / b)
14     else:
15         print("Division by zero is not possible")
16 else:
17     print("Invalid operator")
18
```

OUTPUT:

Output

```
Enter first number: 10
Enter second number: 20
Enter operator (+, -, *, /): +
Result: 30.0
```

2. Write a python program to Add Two Matrices.

AIM:

To write a Python program to add two matrices.

ALGORITHM:

1. Start the program.
2. Initialize two matrices, A and B.
3. Create an empty matrix C to store the result.
4. Add the corresponding elements of A and B using nested loops.
5. Store the sum in matrix C.
6. Display the resultant matrix.
7. Stop the program.

CODE :

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
C = [[0, 0], [0, 0]]
for i in range(2):
    for j in range(2):
        C[i][j] = A[i][j] + B[i][j]
print("Sum of matrices:")
for row in C:
    print(row)
```

OUTPUT:

```
Output

Sum of matrices:
[6, 8]
[10, 12]
```

3. Write a python program to Transpose a Matrix.

AIM:

To write a Python program to find the transpose of a matrix.

ALGORITHM:

1. Start the program.
2. Initialize a matrix A.
3. Create an empty matrix to store the transpose.
4. Use nested loops to interchange the rows and columns.
5. Store the elements in the transposed matrix.
6. Display the transposed matrix.
7. Stop the program.

CODE :

```
A = [[1, 2, 3], [4, 5, 6]]
transpose = []
for j in range(len(A[0])):
    row = []
    for i in range(len(A)):
        row.append(A[i][j])
    transpose.append(row)
print("Transpose of matrix:")
for row in transpose:
    print(row)
```

OUTPUT:

```
Output

Transpose of matrix:
[1, 4]
[2, 5]
[3, 6]
```

4. Write a python program to sort the sentence in alphabetical order?

AIM:

To write a Python program to arrange the words of a sentence in alphabetical order.

ALGORITHM:

1. Start the program.
2. Get a sentence from the user.
3. Split the sentence into individual words.
4. Sort the words in alphabetical order.
5. Join the sorted words into a sentence.
6. Display the sorted sentence.
7. Stop the program.

CODE:

```
1
2  sentence = input("Enter a sentence: ")
3  words = sentence.split()
4  words.sort(key=str.lower)
5  print("Alphabetical order:")
6  print(" ".join(words))
7
```

OUTPUT:

```
Output

Enter a sentence: mathematical operations
Alphabetical order:
mathematical operations
```

5. Write a python program to implement List operations (Nested List, Length, Concatenation, Membership, Iteration, Indexing and Slicing)?

AIM:

To write a Python program to perform list operations such as nested list, length, concatenation, membership, iteration, indexing, and slicing.

ALGORITHM:

1. Start the program.
2. Create a nested list.
3. Find the length of the list using len().
4. Concatenate two lists using the + operator.
5. Check membership using the in operator.
6. Iterate through the list using a for loop.
7. Access an element using indexing.
8. Extract elements using slicing.
9. Display the results of all operations.
10. Stop the program.

CODE:

```
L1 = [10, 20, 30, 40]
L2 = [50, 60]
nested = [L1, L2]
print("List 1:", L1)
print("Nested List:", nested)
print("Length:", len(L1))
print("Concatenation:", L1 + L2)
print("Membership:", 20 in L1)
print("Iteration:")
for item in L1:
    print(item)
print("Indexing:", L1[2])
print("Slicing:", L1[1:3])
```

OUTPUT:

Output

```
List 1: [10, 20, 30, 40]
Nested List: [[10, 20, 30, 40], [50, 60]]
Length: 4
Concatenation: [10, 20, 30, 40, 50, 60]
Membership: True
Iteration:
10
20
30
40
Indexing: 30
Slicing: [20, 30]
```